

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Endowing LLMs with the ability to actively search external knowledge is essential for complex, real-world tasks. Existing approaches fall short: they either lean on prompting to elicit the model’s innate agent capabilities, or hit hard performance ceilings and collapse outright when RL is applied to complex interactive tasks, leaving their agentic potential untapped. We resolve this with **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an RL algorithm that delivers robust agent training through sequence-level optimization and dynamic sample filtering. We train our model purely through RL to interleave multi-turn search and reasoning, eliminating any need for supervised demonstration data. Across multiple QA benchmarks, our 7B model improves over a comparable prior work by **34.1%** and even surpasses the 14B model from prior work on complex multihop QA such as HotpotQA by nearly **9% relative**, all while maintaining exceptional training stability.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) have demonstrated exceptional performance across a spectrum of specialized tasks, including math (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024). Yet one fundamental limitation persists: their knowledge is inherently static, confined to the data on which they were trained. The decisive way to overcome this knowledge cutoff is to equip LLMs with search capabilities, transforming them into agents that actively query external knowledge sources (Jin et al., 2025b). This ability is a canonical instance of tool-calling (Schick et al., 2023), where the model learns to interact with an external search tool to solve problems it cannot answer alone. Mastering this skill demands a complex, multi-step policy, which frames the task as a sequential decision-making problem squarely suited to Reinforcement Learning (RL).

Unlike Supervised Fine-Tuning (SFT), which relies on costly static demonstrations and cannot teach exploration, RL enables LLMs to learn effective policies directly through trial-and-error (Chu et al., 2025). This is precisely why value-free methods like Group Relative Policy Optimization (GRPO) (Shao et al., 2024) have become the dominant paradigm, prized for their simplicity and reduced memory overhead. Yet however well GRPO performs on more constrained tasks, applying it to the open-ended domain of interactive search exposes critical instabilities (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). This fragility traces directly to two fundamental flaws. First, as Zheng et al. (2025) establish, GRPO’s token-level objective is ill-posed when paired with a sequence-level reward, producing high-variance gradients that destabilize training. Second, the sparse rewards inherent to search tasks routinely yield sample groups with homogeneous outcomes (e.g., all successes or all failures), collapsing the advantage signal to zero and leaving no learning signal at all—a failure mode that cripples sample efficiency (Yu et al., 2025; Liu et al., 2025).

We resolve both challenges—instability and inefficient learning—with **Dynamic-filter Sequence-level Policy Optimization (DSPO)**. DSPO synthesizes and sharpens key principles from recent policy optimization research into a single, coherent algorithm. It adopts the sequence-level optimization of GSPO (Zheng et al., 2025) to match the unit of optimization to the unit of reward, aligning the optimization objective with the reward signal and thereby stabilizing the learning process for long-horizon reasoning tasks. On top of this, DSPO introduces a dynamic outcome-based filtering mechanism inspired by DAPO (Yu et al., 2025) that actively constructs training batches from rollout groups containing both successful and unsuccessful outcomes for each prompt, guaranteeing that the advantage signal $\hat{A}_i$ is always effective and stable. Integrating these two components yields a stable, high-performance algorithm purpose-built for complex, multi-turn search and reasoning. Our model delivers a **34.1% relative improvement** over a leading 7B

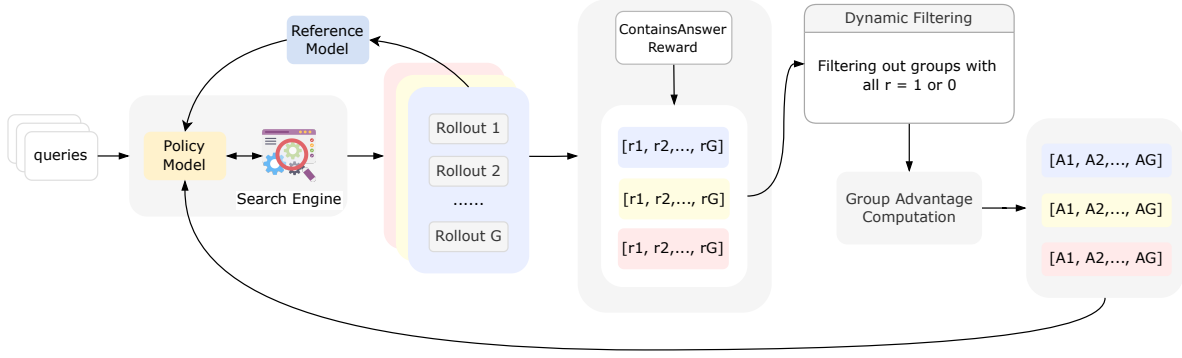


Figure 1: An overview of the DSPO training loop. For a given query, the policy model generates a group of G trajectories by interacting with the search environment, and each trajectory is assigned a sparse terminal reward. The **dynamic filter** discards groups with homogeneous outcomes and keeps sampling until a batch is filled, guaranteeing that every training batch delivers an effective advantage signal. Advantages are then computed and used to update the policy model through the sequence-level objective.

baseline (Jin et al., 2025b) and surpasses even its 14B counterpart (Jin et al., 2025a) on complex multi-hop benchmarks such as HotpotQA, outperforming it by **nearly 9% relative** (0.613 vs. 0.563).

In summary, our main contributions are as follows:

- We propose DSPO, an RL algorithm that overcomes the core instability and sample-inefficiency issues in training agentic search models. It does so by unifying two key principles into a single cohesive framework: **sequence-level optimization** for robust policy updates and **dynamic outcome-based filtering** for a dense and effective learning signal.
- We establish DSPO’s substantial performance gains through rigorous benchmarking. Our 7B model achieves a **34.1% relative improvement** over a comparable 7B baseline and, more strikingly, outperforms its 14B counterpart on complex multi-hop QA, delivering a **nearly 9% relative gain** on HotpotQA (0.613 vs. 0.563).
- We provide extensive empirical evidence for DSPO’s superior training stability, demonstrating that it consistently sustains a stable learning trajectory. Crucially, these results are obtained using only a basic BM25 retriever, isolating the gains squarely to the robustness of our algorithm.

2 RELATED WORK

2.1 RL FOR LLMs

The landscape of RL for LLMs has evolved rapidly, moving from foundational Reinforcement Learning from Human Feedback (RLHF) methods built on PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) to simpler, direct-optimization frameworks like DPO (Rafailov et al., 2023). The decisive shift toward value-free optimization arrived with Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which simplifies training by deriving the reward signal directly from group statistics. GRPO’s token-level objective, however, is a well-documented source of training instability (Liu et al., 2025; Cui et al., 2025), prompting several targeted improvements. GSPO addresses this by shifting to a sequence-level objective that matches the unit of reward (Zheng et al., 2025), while DAPO tackles inefficient learning from sparse rewards with a dynamic outcome-based sampling mechanism (Yu et al., 2025). GMPO, in a similar vein, stabilizes the token-level objective through geometric-mean aggregation to reduce sensitivity to outliers (Zhao et al., 2025). Even so, our experiments show that each of these algorithms still succumbs to training collapse or performance bottlenecks on agentic search. We build directly on this line of work, unifying the principles of sequence-level optimization and dynamic filtering into a single algorithm that overcomes the distinct challenges of training autonomous search agents.

2.2 LLMs WITH AGENTIC RETRIEVAL

To overcome the static knowledge limitations of LLMs, RAG integrates external retrievers that dynamically incorporate evolving information (Lewis et al., 2020; Gao et al., 2023). Classic RAG frameworks employ dense retrievers to

fetch relevant documents, which are then concatenated into the LLM’s input for generation (Karpukhin et al., 2020). Such approaches, however, rest on fixed pipelines that sharply limit autonomy in complex, multi-turn scenarios. Research has since moved decisively toward agentic paradigms, where LLMs act as autonomous agents that plan, search, and reason iteratively. Frameworks like ReAct synergize reasoning and acting to let LLMs interact with tools for tasks such as web navigation (Yao et al., 2023), while multi-agent systems, including AutoGen, orchestrate collaborative workflows (Wu et al., 2024). The most recent advances center on agentic RAG and RL integration, where agents strengthen retrieval through explicit decision-making. Wu et al. (2025) introduce Agentic Reasoning, a framework that integrates external tools for streamlined LLM reasoning, and a family of RL-integrated approaches (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) trains LLMs to interleave reasoning and search through pure RL. This end-to-end paradigm internalizes agent capabilities and sidesteps the engineering overhead of multi-agent frameworks. Yet these methods remain hampered by training instability and performance ceilings in the open-ended search domain. Our work confronts these bottlenecks head-on: DSPO is a robust, efficient training framework that guarantees stable policy optimization and enables LLMs to master effective multi-turn search strategies.

3 METHODOLOGY

In this section, we first formulate agentic search as an RL problem and review prior policy optimization algorithms, pinpointing exactly where they break down in this setting. We then introduce our algorithm, **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, detailing the two core components that secure training stability and training efficiency. Finally, we present the integrated training algorithm.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. Training LLMs via RL commonly relies on policy gradient methods such as PPO (Schulman et al., 2017), the workhorse algorithm for LLM alignment. PPO optimizes a policy π_θ by maximizing a clipped surrogate objective using samples from an old policy $\pi_{\theta_{\text{old}}}$. The objective, averaged over tokens, is:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. PPO, however, requires a separately trained value model to estimate token-level advantages $\hat{A}_t$ via Generalized Advantage Estimation (GAE) (Schulman et al., 2015), which imposes substantial memory overhead and itself becomes a source of instability.

GRPO (Shao et al., 2024) was proposed precisely to remove this burden. It eliminates the value model altogether by sampling a group of G responses $\{y_i\}_{i=1}^G$ for a given prompt x and computing each response’s advantage by normalizing its reward against the group’s statistics. Like PPO, it optimizes the objective at the token level:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

Crucially, every token within a response y_i shares the same advantage $\hat{A}_i$, which is derived from a sequence-level reward—an inherent mismatch that we return to below.

Agentic Search as a Markov Decision Process. We cast the iterative process of agentic search and reasoning as a sequential decision-making problem, formalized as a discrete-time, finite-horizon Markov Decision Process (MDP).

- **State (s_t):** At turn t , the state s_t encodes the entire interaction history: the initial question q , all previously generated thoughts and search queries, and the retrieved evidence returned by the environment. Because

search results must be interpreted and integrated into subsequent decisions, the state grows with the trajectory, creating exactly the long-horizon dependencies that conventional token-level RL fails to optimize.

- **Action (a_t):** An action is a full textual segment generated by the policy π_θ , consisting of free-form reasoning followed by a decision. The action terminates either with a `</tool_call>` token, which triggers a search and returns results within `</tool_response>`, or with a `</answer>` token, which ends the trajectory. This structured action space compels the agent to learn not only *what* to generate but also *when* to search—a demand that introduces substantial variability in trajectory length.
- **Policy (π_θ):** The policy π_θ is the underlying LLM, generating tokens autoregressively conditioned on the state. It conditions on the full history, yet the optimization target is computed only over the model-generated thoughts and actions, masking out the retrieved content from the environment (Jin et al., 2025b). Optimizing this policy demands credit assignment across long sequences in which many intermediate reasoning steps receive no direct supervision or reward—a requirement that makes stable sequence-level optimization not merely helpful but necessary.
- **Trajectory (τ):** A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records all reasoning and tool interactions taken for a given question, spanning both model-generated actions and environment-returned search results. Because the final reward is assigned to the entire trajectory, the optimization problem is fundamentally sequence-level: every early decision directly shapes the eventual answer correctness.
- **Reward ($R(\tau)$):** We adopt a sparse terminal reward: a trajectory receives $R = 1$ if the final answer contains the ground-truth text, and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we explicitly model the search engine $\mathcal{S}$ as part of the environment. The policy LLM π_θ learns to generate trajectories by interleaving reasoning with calls to $\mathcal{S}$. The overall objective is to find a policy that maximizes the expected reward, regularized by a KL divergence term that keeps the policy from drifting far from a reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x; \mathcal{S}) || \pi_{\text{ref}}(y|x; \mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_\theta(\cdot|x; \mathcal{S})$ signifies that the trajectory y is generated through a multi-step process combining the policy’s token generation with the information returned by the search engine $\mathcal{S}$.

Motivation for DSPO. Although frameworks like Search-R1 (Jin et al., 2025b) successfully framed agentic search as an RL problem, conventional algorithms such as PPO and GRPO run into decisive obstacles here. The open-ended search environment amplifies the instability of token-level optimization. The root cause is the fundamental mismatch between the unit of sequence-level reward assignment and the unit of token-level optimization (Zheng et al., 2025): this discrepancy produces high-variance gradient estimates that accumulate over long trajectories and repeatedly culminate in policy collapse. Compounding this, the sparse binary reward means many training batches contain only successful or only unsuccessful trajectories, yielding a degenerate advantage and thus no learning signal at all—which drastically reduces sample efficiency (Yu et al., 2025; Liu et al., 2025). DSPO is engineered to eliminate both failure modes directly.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO delivers two key innovations over prior methods: (1) it performs policy optimization at the sequence level, aligning the training objective with the trajectory-based reward structure, and (2) it incorporates a dynamic filtering mechanism that guarantees every training batch provides a high-quality, non-zero learning signal. Figure 1 depicts the complete training process that integrates these components.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

Building on GSPO (Zheng et al., 2025), we replace the unstable token-level importance ratio with a theoretically grounded sequence-level counterpart. The sequence-level importance ratio $s_i(\theta)$ for a response y_i is the geometric mean of its token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

This length normalization is what drives down variance and keeps $s_i(\theta)$ within a consistent numerical range regardless of sequence length—an essential property for stable clipping.

Gradient Analysis. The gradient analysis below pinpoints exactly why DSPO is more stable. The gradient of the token-level GRPO objective (unclipped) scales each token’s log-probability gradient by a noisy, token-specific weight $r_{i,t}(\theta)$. Our sequence-level objective, by contrast, scales the average log-probability gradient of the entire sequence by a single, far more stable sequence-level weight $s_i(\theta)$:

$$\nabla_{\theta} J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_{\theta} J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (8)$$

By applying a single, holistic correction factor to the entire trajectory, DSPO eliminates the token-level noise accumulation that plagues prior methods, yielding fundamentally more stable training. In parallel, the dynamic filtering mechanism guarantees a well-formed advantage signal $\hat{A}_i$ by constructing every training batch from rollout groups that contain both successes and failures, eliminating wasted samples in sparse-reward environments.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The sparse binary nature of our reward function directly undermines group-based advantage estimation. If all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ collapses to zero or becomes undefined. Such batches yield no useful gradient signal and squander computational resources outright.

DSPO removes this bottleneck with a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, we retain only groups of trajectories that contain a mix of successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ is used for training if and only if its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This forces the reward variance within every training group to be non-zero, guaranteeing a meaningful advantage signal on every update. The dynamic selection curates a high-quality dataset for each policy step, converting a sparse reward problem into a dense and efficient learning signal.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

Integrating these components yields the final DSPO objective. For each valid group from the filtered sample space $\mathcal{D}_{\text{filtered}}$, we compute the advantage $\hat{A}_i$ via group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. The policy π_{θ} is updated by maximizing:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. Crucially, during likelihood calculation we apply loss masking to all tokens retrieved from the search tool, following Jin et al. (2025b). This forces the model to learn how to *use* external knowledge for reasoning rather than merely reproduce it. Algorithm 1 details the full training process.

4 EXPERIMENTS

We now present a series of experiments that empirically establish the effectiveness of Dynamic-filter Sequence-level Policy Optimization (DSPO). The experiments are designed to demonstrate three claims: DSPO (1) achieves exceptional performance on challenging question-answering benchmarks; (2) delivers markedly stronger training stability, avoiding the catastrophic collapse that plagues baseline methods; and (3) owes its gains to the genuine synergy of its two core components.

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```

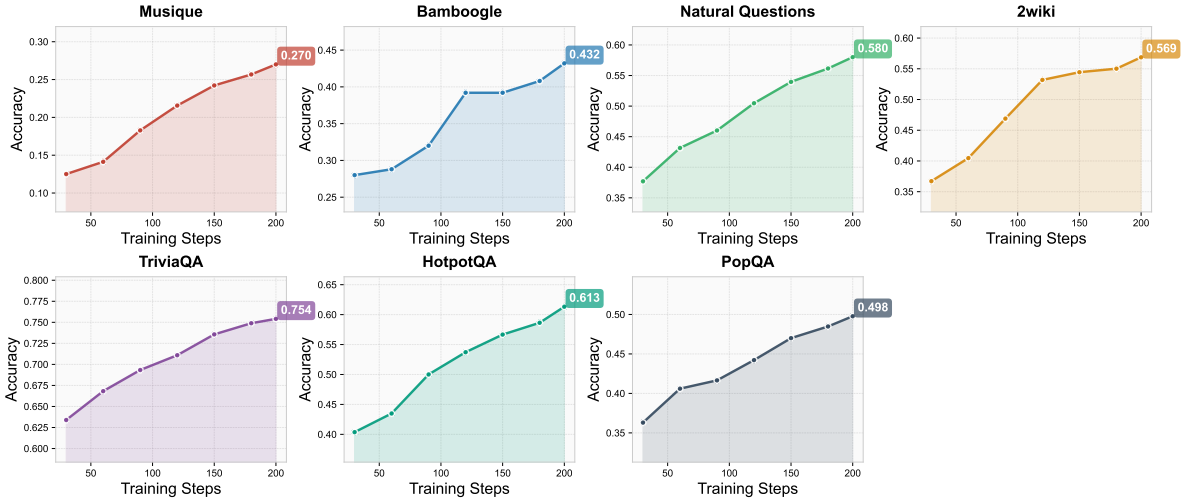


Figure 2: **Validation performance of DSPO across seven benchmarks during training.** The steady, monotonic rise in accuracy is direct evidence that DSPO’s reward improvement translates into genuine generalization, confirming that our method learns a robust search-and-reasoning policy.

4.1 EXPERIMENTAL SETUP

Following Search-R1 (Jin et al., 2025b), and as shown in Table 1, we use a prompt template that instructs the model’s actions during the search task through `<think>`, `<tool_call>`, and `<answer>` tags.

Benchmarks and Baselines. For a rigorous evaluation, our experimental design adheres strictly to the established protocol of Search-R1 (Jin et al., 2025b). We train our model on a composite dataset combining the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018), then measure its generalization on the test sets of seven diverse QA benchmarks: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022).

Our comparison suite pairs strong external baselines with decisive internal ablations. The external baselines are the Qwen2.5-7B and 14B models trained with PPO and GRPO from the Search-R1 framework (Jin et al., 2025b;a). To

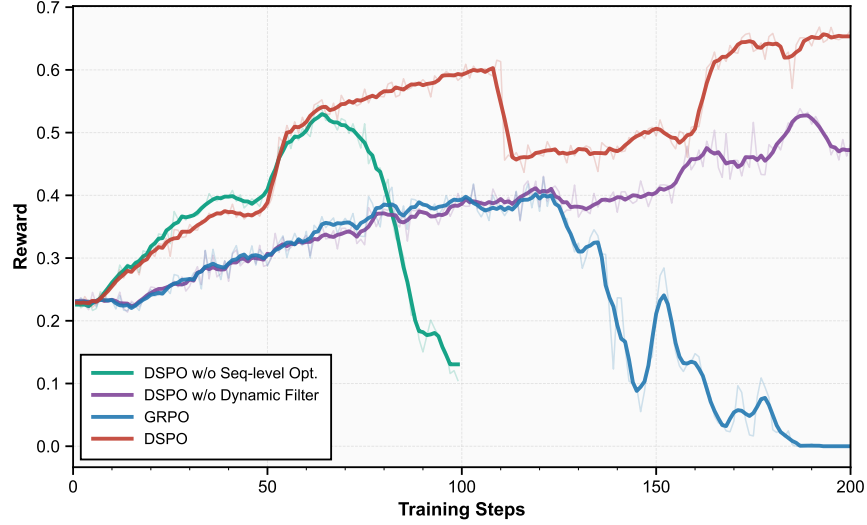


Figure 3: **Training reward dynamics of DSPO and its ablations.** A comparative view of the learning curves. DSPO (red) achieves stable, monotonic improvement throughout training. The token-level variants (green, blue) instead suffer catastrophic policy collapse, and the sequence-level variant without our filter (purple) stalls at a decisively suboptimal level.

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: The prompt template used throughout our experiments.

isolate the contribution of each component, we include two ablations: (1) **DSPO w/o dynamic filter**, equivalent to GSPO (Zheng et al., 2025), and (2) **DSPO w/o sequence-level opt.**, which reverts to the strong token-level policy DAPO (Yu et al., 2025).

For implementation, we use the Qwen2.5-7B-Instruct model as the starting checkpoint for all training. Our experiments build on the VeRL framework (Sheng et al., 2025), adapting its Search-R1-like example code and scripts to our methodology. We benchmark DSPO against a comprehensive suite of baselines, using the PPO and GRPO methods from the Search-R1 framework (Jin et al., 2025b;a) for external comparison. Because our work employs a modified reward function, we retrained these models under our exact experimental conditions to guarantee a fair comparison; these retrained models serve as our primary external benchmarks.

Implementation and Evaluation. To isolate the benefits of our algorithm, all RL experiments deliberately employ a standard BM25 retriever. This controlled setup ensures that every observed improvement is attributable directly to the model’s learned policy rather than to a stronger retriever. Across all methods, models are trained with a sparse, binary reward based on substring Exact Match (subEM) of the final answer, and subEM serves as the primary evaluation metric.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 presents a comprehensive comparison that captures our algorithm’s effectiveness in full. Because DSPO’s components act synergistically, we present the main results alongside the ablation study in a single table. This juxtaposition places DSPO directly against both external state-of-the-art baselines and its own ablated variants, making its overall superiority and the indispensability of each component immediately clear.

Table 2: Comprehensive comparison of DSPO against baselines and ablation variants on seven QA benchmarks. Baselines are the Search-R1 models (7B & 14B) trained with GRPO and PPO (Jin et al., 2025b;a). The ablations remove each key component in turn: ‘w/o dynamic filter’ and ‘w/o seq-level opt.’. Original EM scores from Search-R1 appear in parentheses; for a consistent evaluation, we retrained and re-evaluated these models under our adjusted rewards. Best results are in **bold**; second-best are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	<u>0.754</u>
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

Comparison with Baselines. The results in Table 2 establish DSPO’s clear superiority. Our DSPO-trained 7B agent attains an average score of 0.531, setting a new state of the art—a 34.1% relative improvement over the same-sized Search-R1 (GRPO, 7B) model. More strikingly still, our 7B agent edges out the much larger Search-R1 14B models under both GRPO and PPO. This result is direct evidence that the gains come from a more effective and stable learning algorithm, not from an over-reliance on model scale.

Analysis of Ablations. The ablation results in Table 2 prove unequivocally that both of DSPO’s components are indispensable. Removing the dynamic filter (‘w/o dynamic filter’) triggers a catastrophic drop, with the average score collapsing to 0.313. Without the filter, the sequence-level objective is starved of a useful learning signal by homogeneous-reward batches—a decisive demonstration of the filter’s role. Ablating sequence-level optimization (‘w/o sequence-level opt.’) likewise inflicts a severe degradation, yielding an average of 0.406. Although this token-level variant beats the filter-less one, it falls far short of the full DSPO model and, as the next section shows, remains prone to catastrophic training instability. Together these results confirm that the synergy is essential: sequence-level updates secure stability, while the dynamic filter converts sparse rewards into an efficient learning signal. Neither component alone suffices.

Beyond the quantitative metrics, DSPO gives rise to sophisticated search behaviors—recognizing irrelevant results, reformulating queries, and verifying answers across multiple turns (see Appendix A for detailed trajectory examples). Every one of these behaviors emerges purely from RL training through DSPO.

4.3 ANALYSIS OF TRAINING DYNAMICS

To empirically substantiate our claims about stability and efficiency, we analyze the training reward dynamics of DSPO, its ablations, and key baselines. Figure 3 lays these dynamics bare and directly vindicates our core architectural choices.

DSPO (red) climbs smoothly and monotonically, converging efficiently to the highest reward level—a trajectory that confirms the stability its sequence-level objective delivers. The **token-level methods**—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—stand in stark contrast: both suffer catastrophic policy collapse early in training, their rewards plunging after a brief initial rise, a textbook signature of the instability caused by high-variance token-level gradient updates. **DSPO w/o Dynamic Filter** (purple), which uses sequence-level updates but lacks an efficient learning signal, stays stable yet stalls at a markedly suboptimal ceiling. These dynamics make plain that DSPO’s synergy of sequence-level stability and dynamic filtering is what drives its robust, effective policy optimization.

To confirm that these reward gains reflect genuine generalization rather than reward hacking, we track validation performance on key benchmarks throughout training. As Figure 2 shows, DSPO’s validation accuracy on NQ, HotpotQA, and the other benchmarks rises consistently, mirroring its stable reward curve. This tight correlation confirms that the agent is learning a genuinely generalizable search-and-reasoning policy.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

To further establish the robustness of our approach, we extend the evaluation to model scalability and domain generalization.

Scalability to Larger Models. We test whether DSPO’s stability benefits carry over to larger parameter scales by training Qwen2.5-14B-Instruct. As Table 3 details, DSPO scales cleanly: the DSPO-trained 14B model reaches an average accuracy of 60.6%, decisively outperforming the strong GRPO-14B baseline (53.0%) by a relative margin of **14.3%**. These results confirm that our method fully exploits increased model capacity, consistently exceeding standard baselines at every scale we tested.

Table 3: Scalability analysis on Qwen2.5-14B-Instruct, where DSPO consistently posts the best score on every benchmark. Best results are in **bold**.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. We further probe the generality of DSPO by applying it to single-turn mathematical reasoning tasks with the Qwen2.5 and Qwen3 model families. Table 4 reports the comparison on Math500 and Olympiad-Bench. DSPO surpasses GRPO on both the 7B and 4B model sizes, demonstrating that its benefits extend cleanly beyond search to general reasoning domains.

Table 4: Generalization to mathematical reasoning, where DSPO outperforms GRPO in every setting. Best results are in **bold**.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

In this work, we resolved the critical instability and sample-inefficiency issues that plague RL for autonomous LLM search agents. We introduced Dynamic-filter Sequence-level Policy Optimization (DSPO), an algorithm that secures robust training through two key components: sequence-level optimization that prevents catastrophic policy collapse, and a dynamic outcome-based filter that transforms sparse rewards into a consistently effective learning signal. Our experiments established that DSPO both achieves substantial performance across a suite of challenging question-answering benchmarks and delivers markedly superior training stability over prior methods.

By enabling robust training from environmental feedback alone, DSPO establishes a practical, efficient blueprint for building capable LLM agents without costly expert data. On this stable foundation, future work can confidently integrate advanced retrievers or extend DSPO to complex, multi-tool tasks. Moreover, because sparse rewards and unstable policy gradients are by no means unique to search, we expect DSPO’s principles to deliver comparable performance and stability gains in domains such as mathematics and code generation—a direction our early math results already point toward. We are confident that the core tenets of DSPO—matching the optimization unit to the reward signal and guaranteeing signal density—will be instrumental in developing the next generation of autonomous AI.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilinfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

-
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

To make the learned search strategies concrete, Table A.1 presents representative trajectories from our model. These examples show the model reliably: (1) reformulating queries when initial searches return irrelevant results, (2) verifying information through follow-up searches, and (3) extracting comprehensive answers from the retrieved evidence.

Table A.1: Representative trajectories that showcase our model’s learned search and reasoning capabilities. The model consistently exhibits sophisticated behaviors, including (a) recognition of irrelevant results, (b) query reformulation, and (c) verification strategies.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let’s try searching again with a more specific query."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to Cornell University, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician